LangGraph v1.0.x

Section 01: State, Nodes & Edges

Agentic AI Foundations

March 2026

Three Building Blocks of Every LangGraph App

State + Nodes + Edges

- **State:** The shared data structure (TypedDict)
- **Nodes:** Functions that compute and update state
- **Edges:** Routes that connect nodes together

Every LangGraph application starts here. Master these three, and you can build any workflow.

State Node Edge

State: TypedDict Schema

State is a TypedDict

State defines the *shape* of data flowing through your graph. It's just a Python dict with types.

Example State

```python
from typing_extensions import TypedDict

class State(TypedDict):
    user_input: str
    messages: list
    decision: str
```

Why

Type hints enable validation and IDE autocomplete.

State Fields: Defining Your Schema

Pattern	Use Case
`user_input: str`	Single string field
`messages: list`	Chat history, append-only
`score: float`	Numeric decision
`context: dict`	Nested data
`Optional[str]`	Field can be None

No Inheritance

`TypedDict` does NOT support inheritance. Use composition.

State fields are the interface. Good design = good graph.

Reducers: How State Updates Work

Default Behavior: Overwrite

When a node returns {`"field": new_value`}, the state field is *replaced*.

Default Merge: Overwrite

```
# Node returns partial state
return {"score": 0.85}  # replaces state["score"]
```

Why Reducers Matter

For **lists** (messages, history), you usually want to *append*, not replace. That's where reducers come in.

Annotated: Specifying Reducers

Annotated Pattern

Use `Annotated[type, reducer_function]` to specify custom merge behavior.

Example: List Append

```
from typing_extensions import TypedDict, Annotated
import operator

class State(TypedDict):
    messages: Annotated[list, operator.add]
    score: float
```

What happens

`operator.add` means: `state["messages"] = state["messages"] + new_messages`. Appends!

When a node returns `{"messages": [new_msg]}`, it appends, not replaces.

Built-in Reducers: operator.add

Reducer	Behavior
`operator.add`	`field + new_value` (list concat)
`add_messages`	Smart message appending (dedup, etc)
`Custom`	Define your own merging logic

operator.add Example

```python
class State(TypedDict):
    items: Annotated[list, operator.add]

# Node returns {"items": [4, 5]}
# Result: state["items"] = [1, 2, 3] + [4, 5]
#         = [1, 2, 3, 4, 5]
```

add_messages: The Chat Reducer

add_messages: Most Important Reducer

Built-in reducer from LangChain. Intelligently merges chat messages: handles duplicates, preserves order.

Chat State Example

```
from langchain_core.messages import BaseMessage
from langgraph.graph import add_messages

class State(TypedDict):
    messages: Annotated[list[BaseMessage],
                        add_messages]
```

Why add_messages?

Handles HumanMessage, AIMessage, ToolMessage intelligently. Deduplicates, respects IDs.

Default for LLM workflows.

Complex State: Nested, Optional, Multiple

Advanced State Pattern

```python
from typing_extensions import TypedDict, Annotated
from typing import Optional

class State(TypedDict):
    messages: Annotated[list, add_messages]
    user: Optional[dict]  # Optional field
    metadata: dict  # Nested dict
    scores: Annotated[list[float], operator.add]
```

Guidelines

- Keep state *flat* (avoid deep nesting)
- Use Annotated for lists/accumulators
- Optional for fields that may not exist yet

Nodes: Functions That Update State

Node Signature

A node is a function: `(state: State) -> dict`. Takes state, returns partial updates.

Simple Node

```python
def process_input(state: State) -> dict:
    # Do computation
    result = state["user_input"].upper()
    # Return partial state update
    return {"decision": result}
```

Return Partial

Return ONLY the fields you update. The graph merges with existing state automatically.

Node Return Values

Pattern	Effect
`{field: value}`	Update *that* field (merge)
`{field1: v1, field2: v2}`	Update multiple fields
`{}`	Return empty dict (no update)

Examples

```
return {"score": 0.9}  # Update score only
return {"a": 1, "b": 2}  # Update a and b
return {}  # No state change
```

The graph *merges* your return dict into state. No need to copy full state.

Node Patterns: LLM Calls

LLM Node

```python
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

llm = ChatOpenAI(model="gpt-4")

def llm_node(state: State) -> dict:
    response = llm.invoke(state["messages"])
    return {"messages": [response]}
```

Works with add_messages reducer: appends the response to messages.

Node Patterns: Tool Execution

Tool Node

```python
def tool_executor(state: State) -> dict:
    tool_name = state["tool"]
    tool_input = state["tool_input"]
    result = execute_tool(tool_name, tool_input)
    return {"tool_result": result}
```

Bridges LLM requests and actual execution.

Node Patterns: Routing Logic

Router Node

```
def router(state: State) -> dict:
    if state["score"] > 0.5:
        decision = "approve"
    else:
        decision = "reject"
    return {"decision": decision}
```

Simple conditional logic. (For graph-level routing, use conditional edges-next section.)

Edges: Connecting Nodes

Three Types of Edges

- **Normal edge:** Node A -¿ Node B (always)
- **Conditional edge:** Node A -¿ (B or C or ...) based on state
- **START / END:** Special entry/exit nodes

Edges define the *control flow*. They answer: "After this node, where do we go?"

Normal Edges: add_edge()

Adding Edges

```
graph.add_edge("node_a", "node_b")
graph.add_edge(START, "first_node")
graph.add_edge("last_node", END)
```

START and END

`START` = entry point. `END` = exit point. Graph begins at START, terminates at END.

A simple linear graph: START -¿ A → B → C → END

Conditional Edges: add_conditional_edges()

Router Functions

A router function reads state and returns the *name* of the next node.

Example

```
def route_decision(state: State) -> str:
    if state["decision"] == "approve":
        return "approve_node"
    else:
        return "reject_node"

graph.add_conditional_edges(
    "router",
    route_decision,
    {"approve_node": "approve_node",
     "reject_node": "reject_node"}
)
```

Router returns node name string.

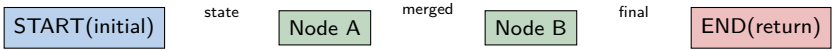
START and END: Special Nodes

Node	Purpose
`START`	Entry point. Initial state flows from here.
`END`	Exit point. Graph terminates, returns final state.

Must Have

- Edge from START
- Edge to END

How State Flows Through Nodes and Edges



Merging at Each Step

1. Node receives *full* state
2. Node returns *partial* dict
3. Graph merges: `state.update(return_dict)`
4. Next node gets updated state

The Graph Lifecycle

Step by Step

```python
# 1. Define state
class State(TypedDict):
    messages: Annotated[list, add_messages]

# 2. Create graph
graph = StateGraph(State)

# 3. Add nodes
graph.add_node("llm", llm_node)

# 4. Add edges
graph.add_edge(START, "llm")
graph.add_edge("llm", END)

# 5. Compile
compiled = graph.compile()

# 6. Invoke
result = compiled.invoke({"messages": []})
```

Common Mistakes

Mistake 1: Returning Full State

```
# WRONG
def node(state: State) -> State:
    return {"a": 1, "b": 2, "c": 3}  # All fields

# RIGHT
def node(state: State) -> dict:
    return {"a": 1}  # Only what changes
```

Mistake 2: Forgetting START/END

```
# WRONG: no entry/exit
graph.add_edge("A", "B")

# RIGHT
graph.add_edge(START, "A")
graph.add_edge("B", END)
```

Common Mistakes: Reducer Issues

Mistake 3: Missing Reducer on List

```python
# WRONG: messages will be replaced, not appended
class State(TypedDict):
    messages: list

# RIGHT: use add_messages reducer
class State(TypedDict):
    messages: Annotated[list, add_messages]
```

Without a reducer, returning `{"messages": [new]}` replaces the entire list!

Summary: State, Nodes, Edges

The Three Pillars

- **State:** TypedDict schema + Annotated reducers
- **Nodes:** Functions (state -¿ dict updates)
- **Edges:** Routes (normal, conditional, START, END)

Key Patterns

- `Annotated[list, operator.add]` for appending
- `add_messages` for chat history
- Return partial dicts from nodes
- Define state before writing nodes

Self-Check Questions

1. What is the purpose of a reducer in state?
2. How does `operator.add` differ from default overwrite?
3. What should a node function return?
4. Why is `add_messages` special for chat?
5. What are the three edge types?
6. What happens if you forget the START edge?
7. How is state merged after a node executes?

Quick Answers: (1) Merges updates (not replace). (2) Appends instead of replaces. (3) Partial dict of updates. (4) Smart chat merge logic. (5) Normal, conditional, START/END. (6) Graph won't start. (7) `state.update(node_return)`.